

버스 API 및 시나리오 구축

1. 버스 API가 제공하는 데이터

1.1 위치정보 API — `getBusPosByRtidList` (주력)

노선 ID 1개로 그 노선에서 운행 중인 모든 차량의 스냅샷을 한 번에 반환한다. 호출 1회에 차량 20~30대분이 담기므로 예산 대비 정보량이 가장 좋다.

갱신 주기 5초 / 평균 응답 100ms / 최대 3MB

차량 식별

필드	내용	비고
<code>vehId</code>	버스 ID	상태 키로 사용
<code>plainNo</code>	차량번호 (서울74사2576)	사용자에게 노출 가능
<code>busType</code>	0 일반 / 1 저상 / 2 굴절	

위치

필드	내용	용도
<code>sectOrd</code>	구간 순번	노선상 진행 위치
<code>sectionId</code>	구간 ID	
<code>sectDist</code>	구간 내 이동 거리(km)	구간 진행률
<code>fullSectDist</code>	정류소 간 거리	구간 속도 계산
<code>rtDist</code>	노선 읍셋 거리(km)	기점부터 누적 → 차량 간 간격 계산
<code>gpsX</code> , <code>gpsY</code>	WGS84 좌표	지도 표시용
<code>posX</code> , <code>posY</code>	GRS80 좌표	

상태 · 예측

필드	내용	용도
<code>stopFlag</code>	0 운행중 / 1 도착	실제 도착 이벤트
<code>nextStId</code>	다음 정류소 ID	예측 대상
<code>nextStTm</code>	다음 정류소 도착 소요시간(초)	예측값
<code>lastStTm</code>	종점 도착 소요시간(초)	장기 리드타임 예측

필드	내용	용도
<code>lastStnId</code>	최종 정류소 ID	
<code>congetion</code>	혼잡도 0 없음 / 3 여유 / 4 보통 / 5 혼잡	실시간. 광역은 잔여좌석
<code>isFullFlag</code>	만차 여부	승차 실패 가능성
<code>isrunyn</code>	운행 여부 (0 종료 / 1 운행)	trip 종료 판정
<code>trnstnid</code>	회차지 정류소 ID	상·하행 판정
<code>islastyn</code>	막차 여부	
<code>dataTm</code>	제공 시각 (20190109160221)	파이프라인 지연 측정

핵심

`nextStTm` (예측)과 `stopFlag=1` (실측)이 한 응답에 함께 있다. 위치 API 단독으로 예측·실측 쌍이 완결되므로 도착정보 API와의 조인 없이 잔차 계산이 가능하다.

```
t0: nextStId=A, nextStTm=362, stopFlag=0
t1: nextStTm=240
t2: nextStTm=95
t3: nextStId=A, stopFlag=1          ← A 도착 확정
```

잔차 = (t3 - t0) - 362

파싱 주의

- `congetion` — congestion이 아니라 **오타 그대로** 온다
- 좌표계 두 벌 (`gpsX/Y` WGS84, `posX/Y` GRS80)
- `resultType=json` 명시 필요 (기본 XML)
- 응답에 상·하행 차량이 섞여서 온다 → `trnstnid` 기준 분리

1.2 도착정보 API (보조)

정류소 관점의 예측을 제공한다. 사용자가 앱에서 실제로 보는 값에 해당한다.

제공 항목

첫 번째·두 번째 도착 예정 버스의 예측 도착 시간(초)

차량번호, 버스유형, 혼잡도, 만차 여부, 막차 여부

주요 정류소별 예정 여행시간

다음 정류소 정보,

배차간격

, 노선 유형

위치 API의 `nextStTm`은 바로 다음 정류소 1구간만 예측하는 반면, 도착정보 API는 여러 정류소 앞선 예측을 준다. 사용자 관점 검증용으로 선정 정류소 몇 곳만 보조 수집한다.

1.3 정적 데이터 (1회 수집 후 캐싱)

데이터	용도
노선 정보 (정류소 순번표)	노선 자동 탐색, 구간 정의
정류소 위치 정보	지도 표시
노선별 운행거리	검증

실시간 호출 예산을 소모하지 않는다. 하루 1회 갱신이면 충분하다.

1.4 제약

항목	값
이용허락	공공누리 1유형 (출처표시)
심의	개발/운영 모두 자동승인
개발계정 트래픽	1,000회/일
운영계정	활용사례 등록 후 증량 신청

노선 1개 × 5초 폴링 × 19시간 = 13,680회/일
전 노선(350개) 5초 폴링 = 약 480만회/일

증량 신청이 크리티컬 패스. 승인 한도가 전체 설계(폴링 주기, 노선 수, 2층 배분)를 결정한다.

2. 연산 가능한 계산

2.1 이벤트 단위 (상태 불필요)

수신 즉시 계산된다.

계산값	산식
파이프라인 지연	<code>received_at - dataTm</code>
예측 절대시각	<code>dataTm + nextStTm</code>
사용자 체감 예측	<code>received_at + nextStTm</code>
시간 버킷	30분 단위, 요일, 평일/주말/공휴일
러시아워 플래그	출퇴근 피크 여부

앞의 두 예측값 차이가 대부분의 앱이 놓치는 오차다.

2.2 차량 궤적 단위 (키: `vehId` + trip)

계산값	방법	의미
예측 스냅샷 리스트	(리드타임, 예측절대시각) 누적	잔차 매칭 재료
예측 수렴 기울기	리드타임 감소당 예측 변화량	0이면 안정, 양수면 계속 밀리는 중
예측 변동성	스냅샷 표준편차	요동치면 불안정
구간 소요시간	<code>stopFlag=1</code> 이벤트 간 시간차	여정 계산의 기본 재료
구간 속도	<code>fullSectDist</code> ÷ 소요시간	정체 판정
정차 시간	<code>stopFlag=1</code> 연속 관측 수 × 폴링주기	5초 폴링 필수
정차 여부	구간 통과 중 <code>stopFlag=1</code> 관측 유무	무정차 통과 판별
trip 경계	<code>trnstnid</code> 통과, <code>isrunyn=0</code>	상·하행 분리
GPS 이상	<code>sect0rd</code> 역행, 순간 이동	필터링

2.3 노선 · 정류소 단위

계산값	방법	의미
차량 간 실제 간격	<code>rtDist</code> 차이 (같은 방향)	앞차 판별
간격 비율	실제 간격 ÷ 계획 배차간격	뭉침 지표
간격 변동계수	간격들의 분산	배차 붕괴 정도
노선 혼잡 계수	다른 차량들의 현재 구간 통과 속도 ÷ 평상시	오늘 이 노선이 밀리는가
구간별 평상시 속도	누적 집계	정체 판단 기준선
정류소별 정차 확률	정차 관측 ÷ 통과 관측	

노선 혼잡 계수가 실시간 반영의 핵심. 내가 기다리는 버스가 아직 멀리 있어도, 같은 노선 앞선 차량들이 방금 그 구간을 어떻게 통과했는지로 현재 도로 상황이 드러난다.

2.4 잔차 (도착 확정 시)

잔차 = 실제_도착시간 - 예측_절대시간
 파이프라인분 = received_at - dataTm (직접 측정)
 관측오차 = $\pm$ 폴링주기 $\div$ 2 (내재)
 운행편차분 = 잔차 - 파이프라인분 (잔여분, 가정)

한 번의 도착이 스냅샷 개수만큼 샘플을 생성한다. 리드타임 10분 잔차와 1분 잔차는 다른 분포를 가지므로 리드타임을 반드시 피쳐로 포함한다.

2.5 분포 상태

키	저장 내용
정류소 $\times$ 방향 $\times$ 시간대	잔차 분포 (t-digest), 관측 건수
구간($i \rightarrow i+1$) $\times$ 시간대	주행 소요 분포, 정차 확률, 정차 시간 분포
노선 $\times$ 시간대	배차간격 분포, 구간 간 상관 계수

페어(A $\rightarrow$ B)별 분포는 유지하지 않는다. 정류소 50개면 페어가 1,000개 넘어 조합 폭발이 발생한다. 구간 분포를 합성해서 임의 페어에 대응한다.

2.6 A $\rightarrow$ B 여정 합성

```

for 정류장 i in A..B:
    정차 여부 ~ Bernoulli(p_i | 시간대, 혼잡도, 앞차간격)
    정차 시간 ~ G_i(· | 혼잡도) if 정차
    주행 시간 ~ H_i(· | 시간대, 노선혼잡계수)

총 소요 =  $\Sigma$  (주행 + 정차)
  
```

몬테카를로로 계산한다. 정차 여부가 이산이라 해석적 합성이 지저분하다. 1만 회 샘플링해도 밀리초 단위다.

상관 처리: 노선 혼잡 계수를 모든 구간이 공유하므로 상관이 자동으로 반영된다. 독립 가정으로 더하면 분산이 심각하게 과소평가된다(도로가 막히면 인접 구간이 함께 늦기 때문).

검증: 관측이 충분한 페어를 홀드아웃으로 남겨, 직접 관측 분포와 합성 분포를 비교해 합성 오차를 정량화한다.

2.7 ML이 개입하는 지점

경험분포로 채울 수 없는 조합 공간을 함수로 메운다.

문제	방법	왜 필요한가
조건부 분위수	LightGBM 분위수 회귀 (q10/q50/q90)	정류소 11,000 × 시간대 × 혼잡도 × 간격 = 조합 폭발
정차 확률	이진 분류	이산 사건, 조건 다수
노선 혼잡 계수	회귀 / 상태공간	노이즈 많은 관측에서 잠재 상태 추정

통계로 충분한 것 — 정류소별 잔차 분포(관측 충분 시), 평상시 배차간격, AOE 누적. 여기에 모델을 넣으면 오히려 나빠진다.

하지 말 것 — 딥러닝(표 형식 데이터는 GBM이 우세), 원천 예측 대체(TOPIS와 경쟁하는 지는 싸움).

평가는 정확도가 아니라 캘리브레이션. "70% 확률"이라 말한 사례가 실제로 70% 정시였는가.

2.8 연산 비용

항목	비용
실시간 추론 (1,400 events/sec)	코어 1개 미만
사용자 조회	DB 읽기 (사전 계산됨)
여정 몬테카를로	밀리초
학습	하루 1회 배치

추론은 병목이 아니다. 제약은 API 호출 예산이다.

3. 시나리오

기반 층 — 정류소 조회

가장 빈번한 사용. 여정 기능의 재료이기도 하다.

강남역 · 152번 (도심 방면)

08:33 도착 예정

1분 이상 빠름	18%
±1분 내 정시	39%
1~3분 연착	31%
3분 이상 연착	12%



08:31 08:33 08:36 08:40

관측 312건 · 이 정류소 고유 분포
서울74사2576 · 3정거장 전 · 여유
이 정류소는 평소 예측보다 평균 48초 늦음

다음 배차 08:41 예정 [예상 기반 · 실시간 반영 전]

연착 구간을 쪼개서 표시한다. "정시 48%"만으로는 부족하고, 늦으면 얼마나 늦는지가 판단에 필요하다.

정시 허용폭은 배차간격에 비례: $\max(60초, \text{계획배차간격} \times 0.2)$

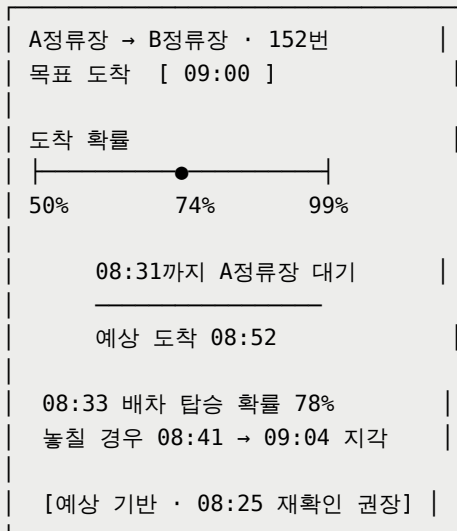
시나리오 1 — 사전 계획 (역산 / 정방향)

상황: 08:05, 집에서. B정류장에 09:00까지 도착해야 함.

내가 탈 버스는 아직 API에 뜨지 않았다.

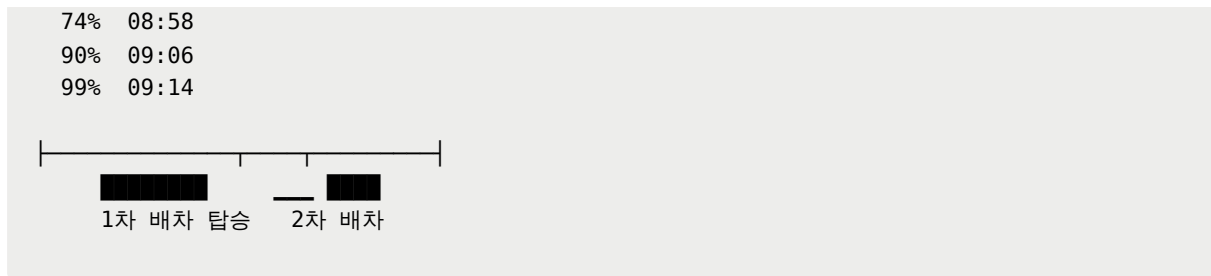
정보원	역할
축적	배차간격 분포, 구간 소요 분포, 정차 확률
실시간	노선 전체 상태 (지금 다니는 다른 차량들의 통과 속도)

실시간 기여가 가장 작은 모드. 그 버스 자체는 모르지만 "오늘 이 노선이 정상인가"는 알 수 있다.



정방향 전환 — 목표 시각을 비우면 "08:31 대기 시 도착 시각 분포"를 보여준다.

10% 08:47 이전
50% 08:54



이봉형이 핵심. 평균 08:56은 실제로 일어나지 않는 값이다. "78% 확률로 08:54쯤, 22% 확률로 09:02쯤"이라는 두 갈래 구조를 그대로 보여준다.

슬라이더 조작 시 출발 시각이 급격히 당겨지는 구간이 배차를 놓치는 경계다. 시각적으로 표시하면 "여기서 1분 늦으면 8분 손해"가 직관적으로 전달된다.

엣지 케이스

- 목표 달성 불가 → 정방향으로 전환해 "지금 나가면 09:07 도착(74%)" 제시
- 데이터 부족 페어 → 구간 합성 근거와 신뢰도 표시
- 막차 이후 → `islastyn` 기반 "마지막 배차 이미 통과"

시나리오 2 — 정류장 대기 중

상황: 08:28, A정류장 도착. 내가 탈 버스가 API에 났다.

정보원	역할
축적	잔차 분포(이 예측이 얼마나 틀리는지), 구간 소요, 정차 확률
실시간	그 차량의 <code>nextStTm</code> , 위치, <code>congetion</code> , 앞차 간격, 노선 상태

실시간 예측에 축적된 잔차 분포를 씌운다.

A정류장 · 08:28 현재

152번 08:33 도착 예정
 탑승 확률 78%
 서울74사2576 · 2정거장 전
 혼잡도 보통 · 앞차와 12분 벌어짐
 ▲ 승객 누적 예상 - 정차 길어질 가능성

B정류장 도착 08:56 (정시 확률 81%)

다음 배차 08:41
 탑승 시 B 도착 09:04 → 목표 초과

분포가 시나리오 1보다 좁아진 것이 화면에서 드러나야 한다. 같은 여정을 두 번 조회했는데 구간이 줄어드는 것이 이 서비스의 성격을 설명 없이 전달한다.

시나리오 3 — 승차 후

상황: 08:34, 152번 탑승 완료.

대기 불확실성이 사라진다. 세 모드 중 계산이 가장 단순하고 정확도가 가장 높다.

정보원	역할
축적	남은 구간 소요 분포, 정차 확률
실시간	이 차량 <code>sect0rd</code> , 혼잡도, 앞차 간격, 남은 구간의 현재 정체

차량 특정 방법

- 기본: 목록에서 선택 (`plainNo` 가 버스 뒷면과 대조 가능)
- 보조: "탔음" 버튼 → 그 시각 A 통과 차량 자동 매칭

서울74사2576 승차 중 · B정류장까지 8정거장

도착 08:54

90% 구간 08:51 ~ 09:01

현재 강남대로 지체 - 앞선 차량 9분 소요(평소 4분)

목표 09:00 도착 확률 82%

운행 중 계속 갱신된다.

승차 시점 도착 08:54 ± 5분
5정거장 후 도착 08:56 ± 3분 (앞 구간에서 2분 소요)
10정거장 후 도착 08:57 ± 1분

시각 경보 — 목표 시각 등록 시 "현재 추세로 09:00 도착 확률 40%로 하락" 알림 가능. 사용자가 중간 하차·택시 전환을 결정할 수 있다. (푸시 인프라 필요 → MVP는 화면 표시 중일 때만 갱신)

3.4 공통 엔진

세 시나리오가 하나의 엔진 위에 올라간다. 조건 벡터에 무엇이 채워졌는지만 다르다.

D(조건벡터) → B 도착 시각 분포

시나리오 1: {노선, A, B, 시각, 노선혼잡계수}

시나리오 2: {... + vehId, nextStTm, 혼잡도, 앞차간격}
시나리오 3: {... + 현재 sectOrd, 대기항 제거}

비어 있는 필드는 축적 분포에서 주변화한다. 모드가 늘어도 엔진은 하나다.

시나리오	실시간 비중	분포 폭	구현 난이도
1 사전	낮음	넓음	중
2 대기	중간	중간	중
3 승차	높음	좁음	낮음

개발 순서는 3 → 2 → 1. 대기 불확실성이 없는 3번이 가장 쉬워, 쉬운 것부터 동작을 확인하며 올라간다.